CS 61B Fall 2025

Midterm 2 Thursday, November 6, 2025

Solutions last updated: 11/10/25

PRINT Your Name: _____

PRINT Your Student ID: _____

PRINT Student name to your left: _____

PRINT Student name to your right: _____

You have 110 minutes. There are 7 questions of varying credit. (100 points total)

Question:	1	2	3	4	5	6	7	Total
Points:	15	16	15	15	9	16	14	100

For questions with **circular bubbles**, select only one choice (there is only one correct answer).

- ○ Unselected option (completely unfilled)
- ◌ Don't do this (it will be graded as incorrect)
- ● Only one selected option (completely filled)

For questions with **square boxes**, you may select one or more choices (select all that apply).

- ■ You can select
- ■ multiple squares
- ☑ Don't do this (it will be graded as incorrect)

- Anything you write outside the answer boxes or you ~~cross out~~ will not be graded. If you write multiple answers or your answer is ambiguous, we will grade the **worst** interpretation.
- Unless otherwise specified, all data structures and algorithms behave according to their implementation in lecture, with no additional optimizations.
- If an implementation detail (e.g. tiebreaking scheme, linked list topology) or a Java library not on the reference card (e.g. Google Truth) is relevant, it will be explicitly noted in the question.
- You may write at most one statement per blank and you may not use more blanks than provided.
- Your answer will be reformatted according to the 61B/61BL style guidelines. For example, any method, constructor, or if-statement requires at least three lines for the purposes of determining line count.
- You may not use ternary operators, lambdas, streams, or multiple assignment.
- Unless otherwise specified, you may assume that everything on the reference card has been imported.

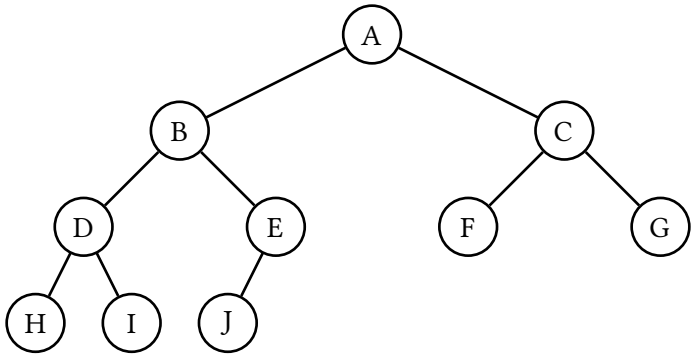
Read the honor code below and sign your name.

> By signing below, I affirm that all work on this exam is my own work. I have not referenced any disallowed materials, nor collaborated with anyone else on this exam. I understand that if I cheat on the exam, I may face the penalty of an "F" grade and a referral to the Center for Student Conduct.

SIGN your name: _____

Q1 *Treevia* **(15 points)**

Consider the tree below. The letters represent unknown integer values, and all 10 values are **unique**.



Q1.1 (3 points) Suppose the tree represents a max heap. Which could be the third-largest value?

☐ A ■ B ■ C ■ D ■ E ■ F ■ G ☐ H ☐ I ☐ J

Solution:

First, note that all 10 values are unique (no duplicates). For simplicity, we'll assume the values are 1 through 10, inclusive.

A: False. A is the largest value, so it cannot be the third-largest value.

B: True. For example, you could assign A=10, B=8, C=9.

C: True. For example, you could assign A=10, B=9, C=8.

D, E, F, G: True. You can assign the node (D, E, F, G) as 8, then its parent as 9, then its grandparent (A) as 10.

H, I, J: False. H has 3 ancestors (D, B, A), which means that there are at least 3 values greater than H. Therefore, H cannot be the third-largest value. The same argument applies for I and J.

Q1.2 (2 points) Suppose the tree represents a binary search tree. Which could be the third-largest value?

☐ A ☐ B ☐ C ☐ D ☐ E ■ F ☐ G ☐ H ☐ I ☐ J

Solution:

The three largest items are C, F, G, since they are the items greater than A.

Out of these three items, F is smaller than C and G. Therefore, F is the third-largest item.

Q1.3 (2 points) Suppose the tree represents a binary search tree. We delete A using Hibbard deletion. Which nodes could become the new root?

☐ B ☐ C ☐ D ☒ E ☒ F ☐ G ☐ H ☐ I ☐ J

Solution:

In Hibbard deletion, the new root could be the predecessor or successor of A.

The predecessor (largest item out of everything smaller than A) is E.

The successor (smallest item out of everything greater than A) is F.

Q1.4 (2 points) Suppose we perform an in-order tree traversal of the subtree rooted in B. Which will be the last node visited?

○ B ○ D ● E ○ H ○ I ○ J

Solution:

The in-order traversal order is H, D, I, B, J, E.

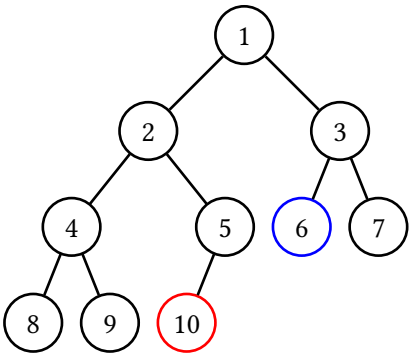
Q1.5 (2 points) The statement "F is strictly greater than J" is always true if the tree represents a _____.

Select all options that could go in the blank.

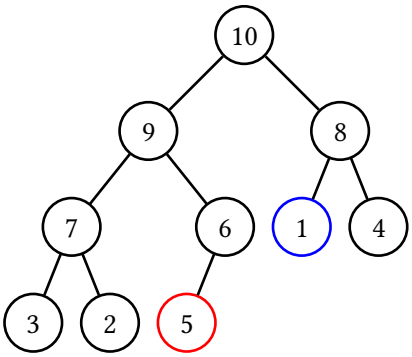
☐ Min heap ☐ Max heap ☒ BST ☐ None

Solution:

A min-heap where F is not greater than J:



A max-heap where F is not greater than J:



In the BST, F > A, and A > J, so we have F > J.

Q1.6 (2 points) Suppose we treat the tree as an undirected graph, and run a breadth first search starting from G. Which could be the last node visited?

☐ A ☐ B ☐ C ☐ D ☐ E ☐ F ☒ H ☒ I ☒ J ☐ None

Solution:

BFS visits nodes in the following order.

Distance 0 from source: G

Distance 1 from source: C

Distance 2 from source: F, A

Distance 3 from source: B

Distance 4 from source: D, E

Distance 5 from source: H, I, J

Since no tiebreaker was specified, any of H, I, J could be the last node visited.

Q1.7 (2 points) Suppose the tree represents a binary search tree, and we want to turn the tree into an LLRB. Select a set of edges to turn red such that the LLRB is valid.

☐ A–B ☒ B–D ☐ C–F ☐ D–H ☒ E–J

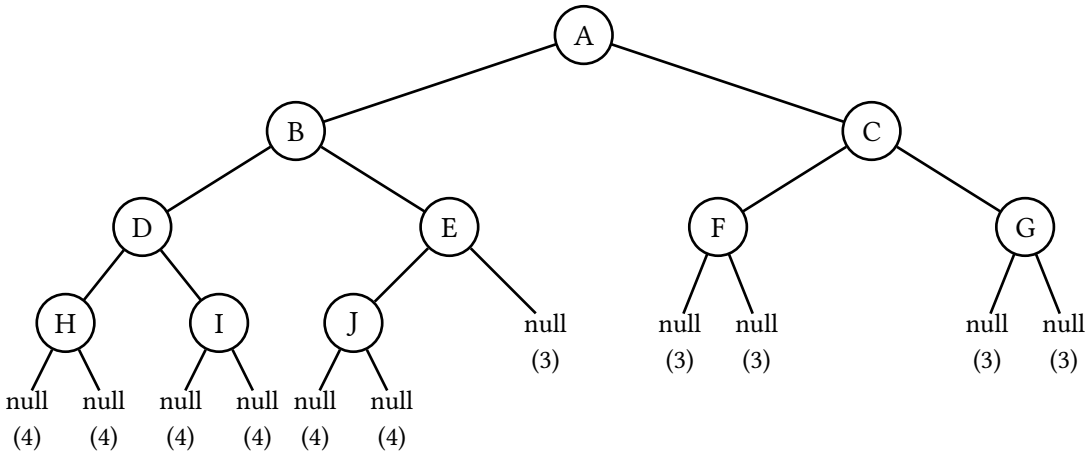
☐ None, the tree is already a valid LLRB with all black links.

Solution:

Recall the LLRB invariant: "The LLRB must have the same number of black links in all paths from root to null (not root to leaf!)"

(See here for more details on root-to-null: https://docs.google.com/presentation/d/1nIKlduNq4WFTV9m6EUz2tOYeZd8D2wAhRI9Nw75VTF0/edit?slide=id.g2bf7cdc29c6_3_0#slide=id.g2bf7cdc29c6_3_0)

This diagram shows the number of black links between every null and the root:



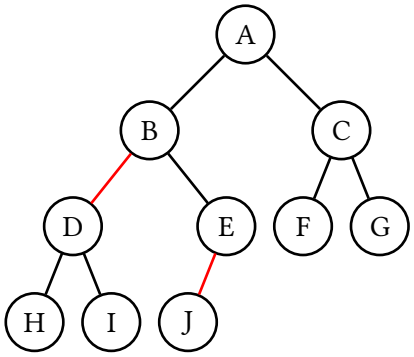
To make this LLRB valid, we need to add red links so that every null is 3 away from the root. (Adding red links can only decrease the number of black links to root, so we can change the 4s to 3s, but we cannot change the 3s to 4s.)

Making B–D red causes all the nulls below D to now be 3 black links from root, instead of 4.

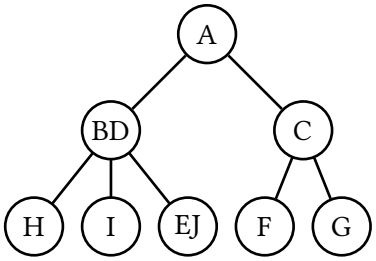
Making E–J red causes all the nulls below J to now be 3 black links from root, instead of 4.

Solution: (continued)

The LLRB after marking red links:



The corresponding 2-3 tree:



Note that the 2-3 tree satisfies both invariants:

1. All leafs are equal distance from the root (i.e. 2 links away from root).
2. Every non-leaf node with k items has $k + 1$ children (e.g. node BD has 2 items and 3 children, and node C has 1 item and 2 children).

There are no other possible answers. If you think you have another answer, try converting it into a 2-3 tree, and you will see that it does not satisfy both invariants.

Q2 *Aditya's Potpourri* **(16 points)**

For Q2.1 and Q2.2, choose whether the following arrays represent a max heap, min heap, or neither:

Q2.1 (1 point) `[-, 1, 7, 2, 8, 15, 5, 3, 17, 9]`

- ● Min heap
- ○ Max heap
- ○ Neither

Q2.2 (1 point) `[-, 59, 38, 47, 26, 46, 45, 13]`

- ○ Min heap
- ○ Max heap
- ● Neither

Q2.3 (2 points) From a pool of $n > 75$ unique integers, Aditya wants to find the 75th-smallest element. Which data structure would be best for this task?

- ○ Hash table
- ○ Linked list
- ● Max heap
- ● Binary search tree

Solution:

This is exactly the example from lecture: https://www.youtube.com/watch?v=iCG9IDkoorY

Here's the $\Theta(n)$ algorithm using a max heap. The max heap always holds the 75 smallest items. For each item:

1. Add the item to the max-heap.
2. If the heap has 76 items, delete the largest item (there are 75 smaller items, so the deleted item is no longer one of the 75 smallest items).

After adding all items, the heap holds the 75 smallest items, so the largest item remaining in the heap is the 75th-smallest item.

Note: A reasonable but strange interpretation of this question was that the n elements were already added to the data structure of choice, i.e. the data structure has already been built. In this case, finding the 75th-smallest element in a binary search tree would simply require an in-order traversal up to 75 items, which can be done in constant time. By contrast, to find the 75th-smallest element in a max heap, we must check the values of all of its $n/2$ leaves, which would be in order of linear time.

As such, we also awarded points to "Binary search tree".

Q2.4 (1 point) True or False: If `hashCode()` is a valid hash function, then `hashCode() + c` is also a valid hash function, where `c` is a fixed integer in $[-1000000, 1000000]$.

● True ○ False

Solution:

Note that in CS61B, we define "valid" to mean that these two conditions are true:
1. Two items that are `equals` always have the same hash code.
2. Calling `hashCode` on the same object (without modifying it) always returns the same value.

These conditions are satisfied for `hashcode() + c`:
1. If two `equals` items have the same `hashCode`, then adding `c` to both hash codes still results in the two items having the same hash code.
2. If the object isn't modified, the hash code stays `hashCode() + c` every time.

Q2.5 (1 point) True or False: If `hashCode()` is a valid hash function, then `hashCode() * c` is also a valid hash function, where `c` is a fixed integer in $[-1000000, 1000000]$.

● True ○ False

Solution:

Similar logic to the previous subpart.

1. If two `equals` items have the same `hashCode`, then multiplying `c` to both hash codes still results in the two items having the same hash code.
2. If the object isn't modified, the hash code stays `hashCode() * c` every time.

Q2.6 (2 points) True or False: Finding a value is asymptotically faster in a 2-3 tree than in an LLRB, because a 2-3 tree has a shorter height.

○ True ● False

Solution:

False. The asymptotic runtime of finding items in 2-3 trees and LLRBs are both $O(\log N)$, so there is no asymptotic improvement.

Q2.7 (2 points) Suppose we have a valid LLRB, and we add an element which creates a right-leaning red link. With no other information, select **all** possible operations that could be the first balancing operation after the insertion.

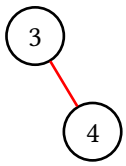
☒ `rotateLeft()` ☐ `rotateRight()` ☒ `colorFlip()`

Solution:

Example of `rotateLeft` being needed:

Valid LLRB before adding:

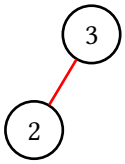
LLRB after adding:



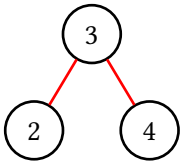
Now a `rotateLeft` is needed to fix the right-leaning red link.

Example of `colorFlip` being needed:

Valid LLRB before adding:



LLRB after adding:



Now a `colorFlip` is needed, because we have a node with two red children.

`rotateRight` is not an answer, because this is only used when there are two consecutive left-leaning red links.

The question says that we start with a valid LLRB (i.e. no consecutive left-leaning red links).

Then, a right-leaning red link is added, which wouldn't cause consecutive left-leaning red links.

Q2.8 (0 points) What is the name of the largest known star?

Stephenson 2 DFK 1

Solution:

Note: this question is worth 0 points and is just for fun.

Correct (as of current consensus Nov 9 2025): Stephenson 2 DFK 1, a.k.a. RSGC2-01, a.k.a. St2-18, a.k.a. “Lil Booper”.

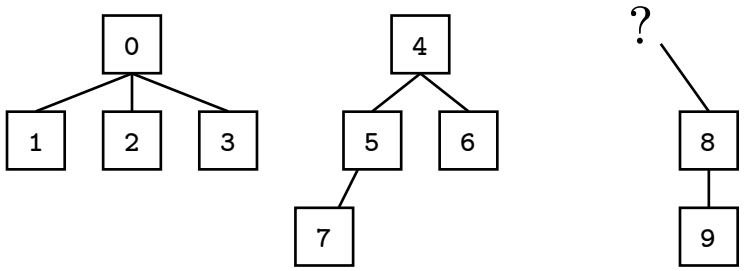
Partial credit: VY Canis Majoris - was once considered the largest but no longer.

Incorrect: UY Scuti - close, but this is in fact the name of Young Thug’s comeback album after winning his RICO trial.

Q2.9 (3 points) Josh built a valid Weighted Quick Union (without path compression) with 10 items. However, he lost one of the values in the parents array!

`[-1, 0, 0, 0, -1, 4, 4, 5, ?, 8]`

(Note: This array uses `-1` instead of `-size` for roots.)



Assuming `8` is not a root, select all items that could be the parent of `8`.

☒ 0 ☐ 1 ☐ 2 ☐ 3 ☒ 4 ☐ 5 ☐ 6 ☐ 7

Solution:

Recall the worst-case tree height from lecture:
https://www.youtube.com/watch?v=xc9s9wdaSdU

H is the height of the tree. N is the minimum number of items needed create a tree of that height.

N	H
1	0
2	1
4	2
8	3
16	4

`1`, `2`, `3` are False, because it would create a tree of height 3 that has 6 items. But from the table, we can see that all trees with height 3 have 8 or more items.

`5`, `6`, `7` are False, because it would create a tree of height 3 (or 4) that has 6 items. But from the table, we can see that all trees with height 3 (or 4) have 8 or more items.

`0` and `4` are True. Imagine if Josh's WQU at an earlier point looked exactly as shown in the diagram above, but with `8` as a root (i.e. no parent). This is a valid WQU (i.e. you could write out a sequence of union operations that creates this WQU). Then, Josh calls either `union(8, 0)` or `union(8, 4)`, which results in a valid WQU where `8`'s parent is `0` or `4`.

Q2.10 (3 points) We are building a Weighted Quick Union (without path compression), and the object initially has zero connections.

What is the minimum number of `union` operations needed to create a tree of height 3?

Recall that a tree's height is the number of links from root to bottommost leaf. For example, a freshly initialized WQU object has a height of 0, not 1.

○ 3 ○ 5 ● 7 ○ 15 ○ 16 ○ 31 ○ 32

Solution:

This is directly from the worst-case tree height demo from lecture:
https://www.youtube.com/watch?v=xc9s9wdaSdU

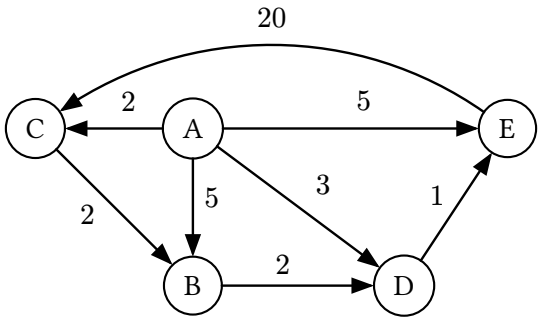
To get a height-1 tree:
- Call `union(0, 1)`.

To get a height-2 tree:
- Call `union(2, 3)` to create another height-1 tree.
- Call `union(0, 2)` to combine the height-1 trees.

To get a height-3 tree:
- Call `union(4, 5)` and `union(6, 7)` to get two more height-1 trees.
- Call `union(4, 6)` to combine the height-1 trees into another height-2 tree.
- Call `union(0, 4)` to combine the height-2 trees.

That's a total of 7 `union` operations.

Q3 *Graphery* **(15 points)**



Suppose we run Dijkstra's on the graph above, starting at vertex A, tiebreaking by alphabetical order.

Q3.1 (2 points) In what order will the vertices be visited?

● A, C, D, B, E ○ A, D, E, C, B ○ A, B, C, D, E ○ A, B, D, E, C

Solution:

Dijkstra's visits vertices in the order of the distance from the source.

Distance 0 from source: A

Distance 2 from source: C

Distance 3 from source: D

Distance 4 from source: B, E

Since we tiebreak alphabetically, we visit B before E.

Q3.2 (2 points) Which vertices have their `distTo` value updated the most number of times?

☐ A ■ B ☐ C ☐ D ■ E

Solution:

All vertices' `distTo` values are initialized to `inf`.

B gets updated twice: `inf` becomes 5 when we relax A –> B, 5 becomes 4 when we relax C –> B.

C gets updated once: `inf` becomes 2 when we relax A –> C.

D gets updated once: `inf` becomes 3 when we relax A –> D.

E gets updated twice: `inf` becomes 5 when we relax A –> E, 5 becomes 4 when we relax D –> E.

Q3.3 (2 points) Suppose we replace the edge weight for A ⟶ B from 5 to −1000. Will running Dijkstra's from A still correctly return a shortest paths tree?

● Yes ○ No

Solution: When you re-run Dijkstra's on the adjusted graph, after we relax A ⟶ B, the distTo B is never less than −1000 (i.e. there is no new path to B that has a lower distance). As a result, the outputted shortest paths tree is valid.

Q3.4 (3 points) Suppose we instead replace the edge weight E ⟶ C. If this edge weight gets too small, there would no longer be a finite shortest path from A to every vertex.

What is the smallest weight for edge E ⟶ C such that there is still a finite shortest path from A to every vertex?

−5

Solution: There exists a cycle in the graph from C ⟶ B ⟶ D ⟶ E. Because E ⟶ C is a part of this cycle, the key is that the total sum of the cycle should never be negative. If this cycle were to be negative, then there would be no shortest path from A to any other vertex (as you could go around the cycle an infinite number of times to reach each vertex). As such, the minimum value of E ⟶ C is −5.

Q3.5 (2 points) Which edge, if reversed, causes the resulting graph to have a valid topological sort?

○ A ⟶ B ○ A ⟶ C ○ A ⟶ D ○ A ⟶ E ● C ⟶ B

○ The graph already has a valid topological sort.

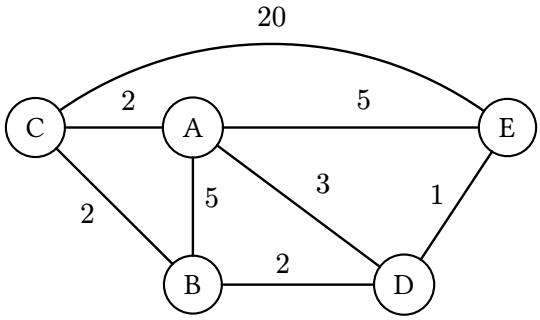
Solution:

Only DAGs (directed acyclic graphs) have valid topological sorts.

The graph has a cycle with edges C → B → D → E → C.

To remove the cycle, we must reverse one of these edges, and C → B is the only one of these edges in the list of answer choices.

Consider the same graph, but with undirected edges.



For Q3.6 to Q3.9, run Prim's algorithm from vertex A, tiebreaking alphabetically.

Q3.6 (0.5 points) First edge added to the MST:

○ A-B ● A-C ○ A-D ○ A-E ○ B-C ○ B-D ○ C-E ○ D-E

Solution: Consider the cut with A on one side, and B, C, D, E on the other side. The smallest edge crossing this cut is A–C.

Q3.7 (0.5 points) Second edge added to the MST:

○ A-B ○ A-C ○ A-D ○ A-E ● B-C ○ B-D ○ C-E ○ D-E

Solution: Consider the cut with A, C on one side, and B, D, E on the other side. The smallest edge crossing this cut is B–C.

Q3.8 (0.5 points) Third edge added to the MST:

○ A-B ○ A-C ○ A-D ○ A-E ○ B-C ● B-D ○ C-E ○ D-E

Solution: Consider the cut with A, B, C on one side, and D, E on the other side. The smallest edge crossing this cut is B–D.

Q3.9 (0.5 points) Fourth edge added to the MST:

○ A-B ○ A-C ○ A-D ○ A-E ○ B-C ○ B-D ○ C-E ● D-E

Solution: Consider the cut with A, B, C, D on one side, and E on the other side. The smallest edge crossing this cut is B–E.

Q3.10 (2 points) For the graph above, the MST outputted by Kruskal's algorithm is ____ the same as the MST outputted by Prim's algorithm (from Q3.6 to Q3.9).

● Always ○ Sometimes (depends on tiebreaking) ○ Never

Solution:

The MST is unique, so regardless of tiebreaker, Prim's algorithm and Kruskal's algorithm only have a single MST that they could output. Therefore, the outputted MST must be the same for both algorithms.

Q4 *Project 4E* **(15 points)**

Project 4 ignored the definitions of words, but Aditya likes definitions. Implement the `WordDefinitions` class below, such that `getDefinitions` returns all definitions of a word. You may not need all lines.

Reminder: `synsetsFile` contains definitions for each synset. Here are some example lines from the file:

```
25,chill iciness,coldness from environment
27,chill shivering,cold feeling
```

Examples based on the lines above:

- `getDefinitions("chill")` should return `["coldness from environment", "cold feeling"]`.
- `getDefinitions("shivering")` should return `["cold feeling"]`.

Hint: `split(String token)` divides a string into an array of substrings on each appearance of `token`.

```
public class WordDefinitions {

    public Map<String, List<String>> definitions;

    public WordDefinitions(String synsetsFile) {

        definitions = new HashMap<>();

        In synsetIn = new In(synsetsFile);

        while (synsetIn.hasNextLine()) {
            String line = synsetIn.readLine();

            String[] splitLine = line.split(",");

            String[] splitWords = splitLine[1].split(" ");

            for (String word : splitWords) {

                if (definitions.get(word) == null) {

                    UNUSED;

                    definitions.put(word, new ArrayList<>());
                }

                definitions.get(word).add(splitLine[2]);

                // UNUSED ;
            }
        }
    }

    /* Assume word exists in the synsets file. */
    public List<String> getDefinitions(String word) {

        return definitions.get(word);

    }
}
```

Solution: Alternate answers:

Blank 1:
- `Map` could be replaced with `HashMap` or `TreeMap`.
- `List` could be replaced with `ArrayList`.

Blank 2:
- `HashMap` could be replaced with `TreeMap`.
- Filling in the generic type is fine, e.g. `new HashMap<String, ArrayList<String>>()`. It's unnecessary, though.

Blanks 3 and 6:
- `List<String>` does not work, because we said in the question that `split` divides a string into an *array* of substrings, not a list of substrings.

Blank 10:
- `definitions.get(word)` could be replaced with `getDefinitions(word)`.

Blanks 11-12:
- The one-line solution `definitions.put(word, new ArrayList<>())` could be replaced with an equivalent two-line solution:

```
List<String> empty = new ArrayList<>();
definitions.put(empty);
```

or

```
ArrayList<String> empty = new ArrayList<>();
definitions.put(empty);
```

Blanks 13-14:
- The one-line solution `definitions.get(word).add(splitLine[2])` could be replaced with an equivalent two-line solution:

```
List<String> defs = definitions.get(word);
defs.add(splitLine[2]);
```

or

```
ArrayList<String> defs = definitions.get(word);
defs.add(splitLine[2]);
```

Alternate solution: Use a set instead of a list to track definitions:
- Blank 1 is `Map<String, Set<String>>`.
- Blank 2 is `new HashSet<>()`.
- The last blank is `return new ArrayList<>(definitions.get(word))`, where we use the ArrayList constructor to convert the set into a list.

Note that the ArrayList constructor wasn't provided on the reference card, since it wasn't our intended solution, but we gave points since it technically works.

Also, note that `definitions.get(word).toList()` would not work, because sets do not have a `toList` method (not on the reference card, and also not in the actual Java library).

Q5 *Hash Table* **(9 points)**

Consider the class below:

```
public class MutableString {
    public String str;

    public MutableString(String s) {
        this.str = s;
    }

    /* Changes the idx'th character of str to c. */
    public void changeChar(int idx, char c) { ... }

    /* Adds character c to the end of str. */
    public void addChar(char c) { ... }

    @Override
    public boolean equals(Object other) {
        if (other instanceof MutableString uddaMutableString) {
            return this.str.equals(uddaMutableString.str);
        }
        return false;
    }

    @Override
    public int hashCode() {
        return this.str.length();
    }
}
```

Suppose we insert four items into a `HashSet` of `MutableString` objects.

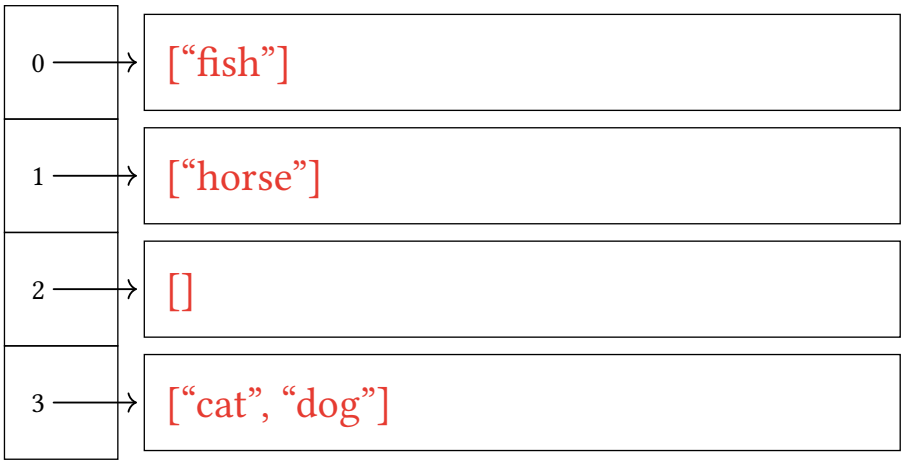
```
Set<MutableString> h = new HashSet<>();

MutableString s1 = new MutableString("cat");
h.add(s1);

h.add(new MutableString("fish"));
h.add(new MutableString("dog"));
h.add(new MutableString("horse"));
```

Q5.1 (3 points) Assuming the hash table has four buckets, draw the results of the insertions above in the hash table. Do not worry about resizing.

You may refer to `MutableString` objects by the value of their `str` field. In each box, write a list like `["cat", "fish", "dog", "horse"]`, or `[ ]` if the bucket is empty.



Solution:

First, note that `equals` compares the strings. All four strings are different, so none of the four objects are equal to each other. Therefore, we will get four distinct objects added into the hash set.

Next, note that `hashCode` returns the length of the string.

- `fish` has hash code 4, which maps to bucket $4 \bmod 4 = 0$.
- `horse` has hash code 5, which maps to bucket $5 \bmod 4 = 1$.
- `cat` has has code 3, which maps to bucket $3 \bmod 4 = 3$.
- `dog` has has code 3, which maps to bucket $3 \bmod 4 = 3$.

Q5.2 (3 points) After the four insertions, we run `s1.changeChar(0, 'b')`, changing `s1.str` from `"cat"` to `"bat"`.

Which of the following are true?

- [x] `h.contains(s1)` returns `true`.
- [x] `h.contains(new MutableString("bat"))` returns `true`.
- [ ] `h.contains(new MutableString("cat"))` returns `true`.
- [ ] None of the above

Solution:

Note that `s1.changeChar(0, 'b')` changes the item in bucket 3 of the hash table from `cat` to `bat`. Therefore, bucket 3 now contains `["bat", "dog"]`.

Any time we call `contains`, the first thing that happens is hashing the object. In all options options, the string has 3 characters, so the hash code is 3. This maps to bucket $3 \bmod 4 = 3$.

Next, in all options, the `contains` method will look in bucket 3 for an equal object.

`s1`'s string is `bat`, which does live in bucket 3, so `h.contains(s1)` returns `true`.

`new MutableString("bat")` has string `bat`, which does live in bucket 3, so `h.contains(new MutableString("bat"))` returns `true`.

`new MutableString("cat")` has string `cat`, which does not live in bucket 3, so `h.contains(new MutableString("cat"))` returns `false`.

Q5.3 (3 points) Consider the hash table from Q5.1, i.e. we have **not** changed `"cat"` to `"bat"`.

Suppose we run `s1.addChar('s')`, changing `s1.str` from `"cat"` to `"cats"`.

Which of the following are true?

- ☐ `h.contains(s1)` returns `true`.
- ☐ `h.contains(new MutableString("cat"))` returns `true`.
- ☐ `h.contains(new MutableString("cats"))` returns `true`.
- ☒ `h.add(new MutableString("cats"))` adds a new element to our hash table.
- ☐ None of the above

Solution:

Note that `s1.addChar('s')` changes the item in bucket 3 of the hash table from `cat` to `cats`. Therefore, bucket 3 now contains `["cats", "dog"]`.

Also, note that bucket 0 still contains only `["fish"]`. Mutating an object does not change its bucket in the hash table.

First option is false. `s1`'s string is `cats`, so the hash code is 4. We look in bucket $4 \bmod 4 = 0$, which does not contain `cats`.

Second option is false. The new object's string is `cat`, so the hash code is 3. We look in bucket $3 \bmod 4 = 3$, which does not contain `cat` (it contains `["cats", "dog"]`).

Third option is false. The new object's string is `cats`, so the hash code is 4. We look in bucket $4 \bmod 4 = 0$, which does not contain `cats`.

Fourth option is true. The new object's string is `cats`, so the hash code is 4. We look in bucket $4 \bmod 4 = 0$, which does not contain `cats`, so we add a new element to our hash table.

Q6 *Asymptotics Analysis* **(16 points)**

Q6.1 (4 points) What is the runtime of `mulch` in terms of N?

```
void mulch(int[] x) {
    int N = x.length;
    if (N <= 1) {
        return;
    }

    int[] left = new int[N/2];
    for (int i = 0; i < N/2; i += 1) {
        left[i] = x[i];
    }

    mulch(left);
    mulch(left);
}
```

Best case:

○ $\Theta(1)$	○ $\Theta(\log(\log N))$	○ $\Theta(\log N)$	○ $\Theta((\log N)^2)$	○ $\Theta\left(\sqrt{N}\right)$
○ $\Theta(N)$	● $\Theta(N \log N)$	○ $\Theta(N^2)$	○ $\Theta(N^2 \log N)$	○ $\Theta(N^3)$
○ $\Theta(N^3 \log N)$	○ $\Theta(2^N)$	○ $\Theta(N!)$	○ $\Theta(N^N)$	○ Infinite loop

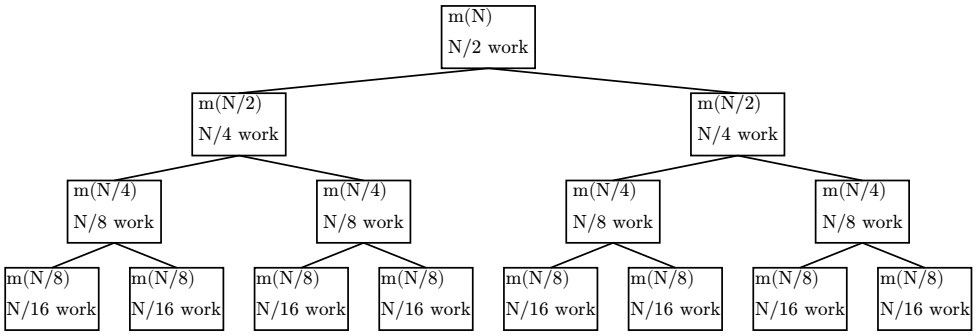
Worst case:

○ $\Theta(1)$	○ $\Theta(\log(\log N))$	○ $\Theta(\log N)$	○ $\Theta((\log N)^2)$	○ $\Theta\left(\sqrt{N}\right)$
○ $\Theta(N)$	● $\Theta(N \log N)$	○ $\Theta(N^2)$	○ $\Theta(N^2 \log N)$	○ $\Theta(N^3)$
○ $\Theta(N^3 \log N)$	○ $\Theta(2^N)$	○ $\Theta(N!)$	○ $\Theta(N^N)$	○ Infinite loop

Solution: Best case: $\Theta(N \log N)$. Worst case: $\Theta(N \log N)$

The recursive tree of calls is shown below.

- Each node represents a call to `mulch` (shortened as `m`).
- Each edge represents a recursive call from parent to child.



Each level does a total of $N/2$ work.

There are $\log(N/2) = \log(N) - 1$ levels. This can be derived by either noting that we start at $N/2$ and divide by 2 at each level until reaching 1, or by noting that the sequence $N, N/2, N/4, \ldots, 1$ has $\log(N)$ items, but we subtract one because we start at $N/2$ instead of N.

This gives a total runtime of $(\log(N) - 1)(N/2)$, and after dropping constants, we get $N \log N$.

Q6.2 (4 points) What is the runtime of **alarmJunior** in terms of N?

```
void alarmJunior(int N) {
    alarmHelper(2, N);
}

void alarmHelper(int i, int N) {
    if (i > N) {
        return;
    }

    System.out.println("BEEP");
    alarmHelper(i*2, N/2);
}
```

○ $\Theta(1)$	○ $\Theta(\log(\log N))$	● $\Theta(\log N)$	○ $\Theta((\log N)^2)$	○ $\Theta\left(\sqrt{N}\right)$
○ $\Theta(N)$	○ $\Theta(N \log N)$	○ $\Theta(N^2)$	○ $\Theta(N^2 \log N)$	○ $\Theta(N^3)$
○ $\Theta(N^3 \log N)$	○ $\Theta(2^N)$	○ $\Theta(N!)$	○ $\Theta(N^N)$	○ Infinite loop

Solution: $\Theta(\log N)$

First, note that each call to **alarmHelper** does $\Theta(1)$ work by printing out a single **BEEP**, so the runtime is equivalent to finding the total number of calls to **alarmHelper**.

Consider the sequence of powers of 2. Note that this sequence has $\log(N)$ numbers in it.

```
2, 4, 8, 16, 32, 64, ..., N/16, N/8, N/4, N/2, N
```

The initial call is **alarmHelper(2, N)**:

```
2, 4, 8, 16, 32, 64, ..., N/16, N/8, N/4, N/2, N
*                                             *
```

This makes a single call to **alarmHelper(4, N/2)**:

```
2, 4, 8, 16, 32, 64, ..., N/16, N/8, N/4, N/2, N
   *                                     *
```

This makes a single call to **alarmHelper(8, N/4)**:

```
2, 4, 8, 16, 32, 64, ..., N/16, N/8, N/4, N/2, N
      *                             *
```

This makes a single call to **alarmHelper(16, N/8)**:

```
2, 4, 8, 16, 32, 64, ..., N/16, N/8, N/4, N/2, N
         *                     *
```

In each subsequent call, the two *s take one step closer to each other, and the code finishes when the two *s meet in the middle.

There are $\log(N)$ items in the sequence, so it will take $\log(N)/2$ steps for the two *s to meet.

Thus, the total number of calls to **alarmHelper** is $\log(N)/2$, and after dropping constants, we get a runtime of $\Theta(\log(N))$.

Q6.3 (4 points) Consider the `BSTMap` from Homework 9. Recall that `get` returns `null` if the item is not in the map.

What is the runtime of `boom` in terms of N, where $N =$ `map.size()`?

Note: The runtime of the `compareTo` method of the `Laboohoo` class is constant.

```
void boom(BSTMap<Integer, Laboohoo> map) {
    for (int i = 0; i < map.size(); i += 1) {
        System.out.println(map.get(i));
    }
}
```

Best case:

○ $\Theta(1)$	○ $\Theta(\log(\log N))$	○ $\Theta(\log N)$	○ $\Theta((\log N)^2)$	○ $\Theta\left(\sqrt{N}\right)$
● $\Theta(N)$	○ $\Theta(N \log N)$	○ $\Theta(N^2)$	○ $\Theta(N^2 \log N)$	○ $\Theta(N^3)$
○ $\Theta(N^3 \log N)$	○ $\Theta(2^N)$	○ $\Theta(N!)$	○ $\Theta(N^N)$	○ Infinite loop

Worst case:

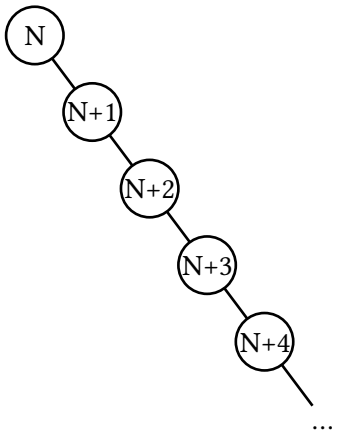
○ $\Theta(1)$	○ $\Theta(\log(\log N))$	○ $\Theta(\log N)$	○ $\Theta((\log N)^2)$	○ $\Theta\left(\sqrt{N}\right)$
○ $\Theta(N)$	○ $\Theta(N \log N)$	● $\Theta(N^2)$	○ $\Theta(N^2 \log N)$	○ $\Theta(N^3)$
○ $\Theta(N^3 \log N)$	○ $\Theta(2^N)$	○ $\Theta(N!)$	○ $\Theta(N^N)$	○ Infinite loop

Solution: Best case: $\Theta(N)$. Worst case: $\Theta(N^2)$

The key idea in this question is that we try and get items $1, 2, 3, \ldots, N$ from the map, but the items in the map are not necessarily $1, 2, 3, \ldots, N$. All we know is that there are N items in the map, but not necessarily what they are.

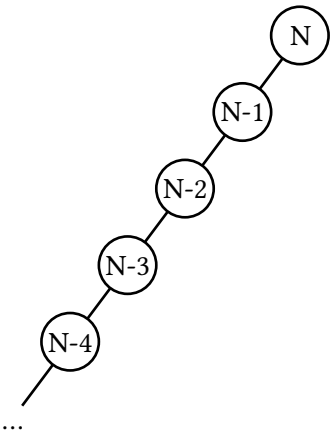
Best case:

Tree containing N, N+1, N+2, ..., 2N-1.



Worst case:

Tree containing N, N-1, N-2, ..., 1.



Solution: (continued)

Best case: Every call to `get` takes $\Theta(1)$ time, since you go left from the root and immediately fall off the tree. There are N calls, for a total runtime of $\Theta(N)$.

Worst case:

- `get(1)` requires traversing N links.
- `get(2)` requires traversing $N-1$ links.
- `get(3)` requires traversing $N-2$ links.
- ...
- `get(N-2)` requires traversing 2 links.
- `get(N-1)` requires traversing 1 link.

The total links traversed across all calls is $N + (N-1) + (N-2) + ... + 2 + 1 = \Theta(N^2)$.

Q6.4 (4 points) When Edsger Dijkstra published his original version of Dijkstra's algorithm, he did not use any special data structure to store the fringe.

Instead, he simply scanned the `distTo` array, choosing the vertex with the smallest value that was not yet `marked` as previously visited.

Give a tight O bound, in terms of V and/or E, for the runtime of this algorithm on a graph with V vertices and E edges. Simplify your answer.

Assume we are using an adjacency list.

$O(V^2)$

Solution:

The `distTo` array is initially all infinity except for one 0.

At each iteration, we spend $\Theta(V)$ time finding the closest unmarked vertex `a`.

We then spend $O(V)$ time updating the distances based on relaxation of `a`'s edges. This is because each relaxation takes $\Theta(1)$ time to update an entry in the `distTo` array, and there are at most V edges coming out of `a` (at most one edge to every vertex).

This entire process is repeated V times, for a total runtime of $O(V^2)$.

Note that there are no log terms in the runtime anymore, because all the logarithmic terms in Dijkstra's come from the priority queue, which we've removed.

Q7 BorBders (14 points)

Bob loves walking between countries, which are represented by the class below:

```
public class Country {
    public String name;           // Name of the country.

    public List<Country> borders; // A list of bordering countries.
                                  // borders is never null.
}
```

Borders go both ways, i.e. if `countryA` appears in `countryB`'s `borders`, then `countryB` appears in `countryA`'s `borders`.

Implement the `WalkManager` class, which has two methods:

- `public WalkManager(List<Country> countries)`:
  Constructor. Given the list of all N countries in the world, prepare any useful instance variables.
  Must run in $O(N^2 \log N)$ time.

- `public boolean canWalk(Country a, Country b)`:
  Returns whether country `a` is reachable from country `b`, using one or more borders.
  Must run in $O(\log N)$ time.

Here is an example in a world that has $N = 4$ countries:
- `USA` borders `[Canada, Mexico]`.
- `Canada` borders `[USA]`.
- `Mexico` borders `[USA]`.
- `Ethanland` borders `[]` (empty list).

```
WalkManager wm = new WalkManager(List.of(USA, Canada, Mexico, Ethanland));

// true: Canada to USA to Mexico.
wm.canWalk(Canada, Mexico);

// false: No sequence of borders exists between Ethanland and USA.
wm.canWalk(Ethanland, USA);
```

The skeleton code begins on the next page.

Implement **WalkManager** according to Bob's requirements.

Reminder: Everything on the reference card has been imported, and all data structures behave according to their implementations in lecture. You may not need all lines.

```
public class WalkManager {
    // Add any instance variables below (if necessary)


    Map<Country, Integer> wquIndex;

    WeightedQuickUnion wqu;

    public WalkManager(List<Country> countries) {

        int N = countries.size();


        wquIndex = new HashMap<>();

        wqu = new WeightedQuickUnion(N);

        for (int i = 0; i < _N_; i += 1) {

            Country curr = countries.get(i);

            wquIndex.put(curr, i);
        }

        for (Country curr : countries) {

            for (Country c : curr.borders) {

                wqu.union(wquIndex.get(curr), wquIndex.get(c));
            }
        }
    }

    public boolean canWalk(Country a, Country b) {


        return wqu.isConnected(wquIndex.get(a), wquIndex.get(b));
    }
}
```

Solution:

Graph-based solutions like `Map<Country, List<Country>> adjList` will not work, because they cannot efficiently handle cases of traversing multiple borders. For example, consider the Canada-to-USA-to-Mexico example given in the question.

You have reached the end of the exam! Nothing on this page is worth any credit.

Playing Favorites

CS61B staff has a collective favorite data structure! What do you think it is?

Hashmaps

Feedback

Leave any feedback, comments, and/or drawings in the box below!